

Interim Project Report

Optimal Spaced Repetition Scheduling via Stochastic Shortest Path MDPs

Sanjida Nasreen, Morgan Lan

Introduction

The project tackles spaced repetition scheduling: given a flashcard or vocabulary item, when should we review it next? Review too soon and we waste effort on something we already know. Review too late and we have already forgotten, now we have to relearn from scratch, which costs $3\times$ more effort. The goal is to find the schedule that gets an item to “mastered” status at minimum total effort.

We model this problem as a stochastic shortest path (SSP) MDP, a variant where we want to minimize total effort until we hit an absorbing goal state (mastery, defined as memory half-life ≥ 360 days). In the memory model (DHP), memory strength is tracked as a single number: the half-life, meaning “days until we have a 50% chance of recalling this.” After a successful review, memory strength grows. After forgetting, memory strength shrinks and difficulty increases.

But there is a second question that matters just as much for real deployment: *does the optimal policy have to be complicated?* A student is not going to consult a 95-row lookup table before every study session. So we also test whether simple, one-sentence rules like “review whenever our recall probability drops below 85%”, can get close to optimal. We call the performance gap the “price of interpretability.”

1 Background

1.1 The Forgetting Curve

The observation that memory decays over time goes back to Ebbinghaus in 1885. Modern spaced repetition systems use the *half-life* formulation: if our current half-life for an item is h days, then after t days without review, our probability of remembering it is:

$$P(\text{recall}) = 2^{-t/h}$$

So if $h = 7$ days and you wait 7 days, you have a 50% chance of remembering. Wait 14 days, and it drops to 25%. The key insight that makes spaced repetition work is that h is not fixed, it changes based on what happens at each review. Successfully recalling an item makes the memory more durable (higher h), while forgetting weakens it (lower h). This is why well-timed reviews produce a “staircase” effect where each successful review pushes the half-life higher.

1.2 The DHP Memory Model

The specific model we use comes from Ye et al. [1] and Su et al. [2], who call it the DHP (Difficulty–Half-life–Probability) model. They fitted it on 220 million learning interactions from MaiMemo, a Chinese language-learning app with around 4 million users. The model has two update formulas, one for when the learner remembers, one for when they forget, with six constants that were calibrated against this data. We use those fitted constants directly in our MDP rather than re-fitting them ourselves. This means our transition probabilities reflect real human forgetting behavior, not made-up numbers.

1.3 Prior Work

Several groups have tried to optimize spaced repetition scheduling. Tabibian et al. [4] at MPI modeled it as a continuous-time stochastic optimal control problem and derived review policies using Hamilton–Jacobi–Bellman equations. Settles and Meeder [3] at Duolingo built the half-life regression model and

released an open-source dataset of 13 million learner–item interactions. Ye et al. [1] formulated it as a stochastic shortest path problem (SSP-MMC) and deployed their solution in production at MaiMemo. What none of these papers do is ask whether a *simple* policy can substitute for the optimal one. They all focus on finding the best schedule. Our project asks a different question: once we have the optimal schedule, how much do you lose by simplifying it into a rule a human can actually follow?

2 Model Specification

We formulate this as a stochastic shortest path MDP for a single vocabulary item. The transition dynamics use the DHP memory model from Ye et al. [1], whose parameters were fitted on 220 million learning interactions from the MaiMemo language app.

Our model is a deliberate simplification of the original SSP-MMC system, which uses 152 halflife bins $\times$ 18 difficulty levels = 2,736 states and a C++ solver that runs 200,000 iterations. We shrink the state space so we can inspect every entry of the optimal policy table easily. The whole point is to understand the policy’s structure, not just compute it.

2.1 State Space

A state is $s = (h, d)$, where h is the memory half-life (days until $P(\text{recall}) = 50\%$) and d is item difficulty. We discretize h into 19 log-spaced bins:

$$\mathcal{H} = \{0.5, 1, 1.5, 2, 3, 4, 6, 8, 12, 16, 24, 32, 48, 64, 96, 128, 192, 256, 360\} \text{ days}$$

and use 5 difficulty levels $\mathcal{D} = \{1, 3, 5, 7, 10\}$. This gives $19 \times 5 = 95$ regular states plus one absorbing terminal state s^* (mastery, when $h \geq 360$). Total: $|\mathcal{S}| = 96$.

The bins are spaced logarithmically because the difference between a 1-day and 2-day half-life is huge (that word is barely in our memory vs. somewhat stable), while the difference between a 300-day and 310-day half-life is negligible.

2.2 Action Space

The scheduler picks a review interval $a \in \mathcal{A} = \{1, 2, 3, 5, 7, 10, 14, 21, 30, 45, 60\}$ days ($|\mathcal{A}| = 11$). These values reflect realistic scheduling granularity, nobody schedules a review for exactly 37 days from now.

2.3 Transition Dynamics

After waiting a days, recall probability follows the exponential forgetting curve $p = 2^{-a/h}$. Two outcomes:

Recall (prob. p): the learner remembers, and memory strengthens. The half-life grows:

$$h' = h \cdot \left(1 + e^{3.81} \cdot d^{-0.534} \cdot h^{-0.127} \cdot (1 - p)^{0.97}\right)$$

Difficulty stays the same. Notice the $(1 - p)^{0.97}$ term: when recall probability is *low* (you barely remembered), memory grows *more*. This makes intuitive sense, a difficult retrieval is a stronger learning signal than an easy one. But as we will see, it also creates non-obvious structure in the optimal policy.

Forget (prob. $1 - p$): the learner does not remember. Memory weakens:

$$h' = e^{-0.041} \cdot d^{-0.041} \cdot h^{0.377} \cdot (1 - p)^{-0.227}$$

Difficulty also increases: $d' = \min(d + 1, d_{\max})$. The item just got harder.

The six fitted constants (3.81, −0.534, −0.127, 0.97, −0.041, 0.377, −0.227) come from calibrating the DHP model against 220 million learning records from MaiMemo [1, 2]. The updated h' maps to the nearest bin; $h' \geq 360$ transitions to s^* .

Worked example. Suppose $h = 8$ days, $d = 5$, and we schedule a review after $a = 7$ days. Then $p = 2^{-7/8} = 0.54$. There is a 54% chance the learner recalls, and if they do, the half-life jumps to $h' \approx 25.3$ days (a $3\times$ increase). If they forget, the half-life drops to $h' \approx 2.9$ days and difficulty increases to $d' = 6$.

2.4 Cost Function

Each review costs:

$$c(s, a) = p \cdot c_{\text{recall}} + (1 - p) \cdot c_{\text{forget}}$$

with $c_{\text{recall}} = 3$ and $c_{\text{forget}} = 9$, following [1]. Forgetting costs $3\times$ more because relearning, re-presenting the item, processing the error, restarting the strengthening process, is substantially more effortful than reinforcing something you already remember. The terminal state s^* has zero cost.

2.5 Optimality Criterion

We minimize expected discounted cost to reach s^* :

$$V(s) = \min_{a \in \mathcal{A}} \left[c(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s') \right], \quad V(s^*) = 0, \quad \gamma = 0.99$$

The original SSP-MMC uses $\gamma = 1$ (undiscounted SSP). We use $\gamma = 0.99$ because:

- It connects directly to the discounted infinite-horizon framework from class.
- It guarantees convergence of value iteration without needing a separate proof that all policies are *proper* (reach s^* with probability 1).
- At $\gamma = 0.99$, the practical difference from the undiscounted solution is negligible, a cost 100 steps in the future is worth $0.99^{100} \approx 0.37$ of face value.

3 Solution Methods

3.1 Value Iteration

We solve the MDP using standard value iteration. The transition matrices are precomputed: P has shape (11, 96, 96) for transition probabilities and C has shape (11, 96) for expected costs. At each iteration, we compute the Q-function for all state-action pairs:

$$Q(s, a) = C(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s')$$

then update $V(s) = \min_a Q(s, a)$ and $\pi(s) = \arg \min_a Q(s, a)$. This is vectorized with NumPy’s `einsum`, so each iteration touches all 96 states and 11 actions simultaneously. The algorithm converged in **76 iterations** with Bellman error below 10^{-6} , taking under one second.

3.2 Policy Evaluation

For each interpretable policy π , we compute its value function by iteratively solving:

$$V^\pi(s) = c^\pi(s) + \gamma \sum_{s'} P^\pi(s'|s) V^\pi(s')$$

until convergence. This gives the analytical expected cost under each policy, complementing the simulation estimates.

3.3 Interpretable Threshold Policies

We define a family of simple policies that can each be stated in one sentence:

Policy	Rule	Parameters
Fixed Interval	“Review every k days, no matter what.”	$k = 3$ or 7
Recall Threshold	“Wait until $P(\text{recall})$ drops to p^* , then review.”	$p^* \in \{0.7, 0.8, 0.9\}$
Difficulty-Aware	“Like threshold, but harder items use a tighter target.”	base $p^* = 0.85$
Tiered Rules	“ $h < 2\text{d}$: 1 day; $h < 8\text{d}$: 3 days; $h < 32\text{d}$: 7 days; ...”	5 tiers

Table 1: Interpretable policies. Each can be explained in one sentence.

The recall threshold policy works by computing the interval as $a = -h \cdot \log_2(p^*)$. So if our half-life is 10 days and our target is $p^* = 0.9$, you wait $-10 \cdot \log_2(0.9) \approx 1.5$ days. Stronger memory $\rightarrow$ longer wait. The difficulty-aware variant adjusts the target: $p^* = 0.85 + 0.01 \cdot (10 - d)$, so harder items get reviewed sooner.

3.4 Monte Carlo Simulation

We complement analytical evaluation with simulation. For each policy, we run 500 independent episodes per difficulty level (2,500 total). Each episode: start with a new item at the given difficulty, follow the policy’s schedule, simulate recall/forget outcomes stochastically, and terminate when $h \geq 360$ (mastery) or 2,000 days elapse. We record total cost, reviews, forgetting events, average recall probability, and days to mastery.

4 Results

4.1 Optimal Policy Structure

Table 2 shows the optimal review interval for selected states. The general pattern is intuitive: when half-life is short (weak memory), the optimal action is to review soon; when half-life is long (strong memory), the scheduler waits longer.

Halflife	$d = 1$	$d = 3$	$d = 5$	$d = 7$	$d = 10$
0.5 days	1d	1d	1d	1d	1d
2 days	1d	1d	1d	1d	1d
8 days	1d	1d	2d	3d	2d
32 days	5d	5d	7d	7d	10d
64 days	5d	10d	14d	3d	21d
128 days	14d	30d	14d	5d	21d
256 days	7d	14d	21d	21d	30d

Table 2: Optimal review intervals for selected states. Full table has $19 \times 5 = 95$ entries.

4.2 Monotonicity Analysis

we might expect the optimal policy to be monotone: longer intervals for stronger memory, shorter intervals for harder items. We checked. It is **not fully monotone**, there are 26 violations in the halflife direction and 25 in the difficulty direction.

For example, at $d = 1$: the interval goes from 21 days at $h = 96$ to 14 days at $h = 128$ to 10 days at $h = 192$ to 7 days at $h = 256$. That is the *opposite* of what we would expect, the scheduler reviews *more often* as memory gets stronger.

The reason is the $(1-p)^{0.97}$ term in the recall update. When recall probability is low (we barely remembered), the memory strengthening effect is *larger*. So in certain states, the optimal policy deliberately lets recall probability drop lower before reviewing, because the payoff in half-life growth more than compensates for the slightly higher risk of forgetting. This is a genuinely non-trivial structural feature that would not show up in a simpler linear memory model.

4.3 Policy Comparison

Table 3 presents the main results.

Policy	Avg Cost	Reviews	Recall%	Mastery%	Days	Δ Cost
Optimal (VI)	35.1	8.7	84.7%	100%	71	—
Difficulty-Aware	36.0	8.9	84.4%	100%	98	+2.7%
Threshold $p^*=0.9$	36.9	9.4	85.4%	100%	96	+5.3%
Threshold $p^*=0.8$	37.1	8.4	77.5%	100%	125	+5.8%
Threshold $p^*=0.7$	41.0	8.4	69.3%	100%	133	+17.0%
Tiered Rules	45.0	9.1	70.2%	100%	101	+28.4%
Fixed 3-day	136.2	28.9	80.0%	100%	87	+288.6%
Fixed 7-day	1783	201	16.2%	44.3%	663	+4987%

Table 3: Simulation results (500 episodes per difficulty level, 2,500 per policy).

4.4 The Price of Interpretability

The central result: well-designed interpretable policies achieve near-optimal performance. Formally, the price of interpretability for policy π is:

$$\Delta(\pi) = \frac{V^\pi - V^*}{V^*}$$

where V^* is the optimal value and V^π is the value under policy π , averaged across starting states.

The **difficulty-aware threshold** (“review at $\sim 85\%$ recall, sooner for harder items”) costs only **+2.7%** more than optimal. That is remarkably small, we lose almost nothing by switching from a 95-entry lookup table to a one-sentence rule. Even the crude **tiered rules** with five fixed categories cost only +28.4% more.

In contrast, naive scheduling fails catastrophically. Fixed 7-day scheduling achieves only 44.3% mastery and costs $\sim 50\times$ more than optimal. This tells us that *some* form of adaptation to memory state is essential, but a simple threshold captures nearly all the benefit of full optimization.

4.5 Difficulty Breakdown

The gap between optimal and threshold policies widens for harder items. At $d = 1$ (easy), the threshold policy nearly matches optimal. At $d = 10$ (hard), the gap is larger because the nonlinear DHP dynamics make the optimal scheduling pattern more complex at high difficulty. However, even at $d = 10$, the difficulty-aware threshold stays within reasonable overhead, because its rule explicitly accounts for difficulty by tightening the recall target.

5 Connection to Course Material

The project extends the course material in several ways. First, it applies stochastic shortest path (SSP) concepts from class to a new domain, human memory modeling, where the absorbing terminal state represents mastery of an item. Second, the project uses discretization of a continuous memory-strength

variable (half-life) within a nonlinear data-driven transition model. While discretization concepts were discussed in class, the DHP memory dynamics introduce additional approximation and modeling complexity beyond the standard tabular examples studied in coursework. Third, the project studies interpretable scheduling policies and quantifies the “price of interpretability,” connecting classical MDP optimization with modern research themes in explainable and human-centered AI.

More concretely: value iteration, Bellman equations, and policy evaluation are used exactly as taught. The monotonicity analysis connects to the structured policies topic, we expected the optimal policy to have monotone structure, and the finding that it does not (due to the nonlinear DHP model) is itself a result about when classical structural results do and do not apply.

6 Discussion

Why monotonicity fails. In class we learned that many MDPs have monotone optimal policies, when one state variable increases, the optimal action moves in a predictable direction. Our model breaks this because the DHP recall update has a $(1 - p)^{0.97}$ term that rewards *difficult* retrievals. A linear memory model (where half-life just grows by a fixed amount after each recall) would produce a perfectly monotone policy. The nonlinearity in the DHP model, calibrated from real data, introduces structure that simple monotone rules cannot capture. This is worth noting because it shows that using realistic transition models can change qualitative properties of the optimal policy, not just the numbers.

Practical implications. For anyone building a spaced repetition system, our results suggest a two-tier approach. For an initial deployment, a simple threshold rule (review at 85–90% recall) captures most of the benefit with essentially zero implementation complexity. We do not need an MDP solver. The full optimization is worth pursuing only if the marginal 3–5% improvement justifies the engineering effort, and even then, the optimal policy is harder to explain to users, which matters in educational settings where trust and transparency affect compliance.

Limitations. We model a single item in isolation. Real systems schedule across thousands of items with a limited daily review budget, creating a constrained MDP where items compete for the learner’s time. Our discretization is also coarser than the original SSP-MMC (96 vs. 2,736 states), which may smooth out some structure. Future work could extend to the multi-item constrained setting, which would require different solution techniques (e.g., Lagrangian relaxation or constrained MDPs).

References

- [1] Ye, J., Su, J., & Cao, Y. (2022). A stochastic shortest path algorithm for optimizing spaced repetition scheduling. *Proc. 28th ACM SIGKDD Conf. on Knowledge Discovery and Data Mining*, 4381–4390.
- [2] Su, J., Ye, J., Nie, L., Cao, Y., & Chen, Y. (2023). Optimizing spaced repetition schedule by capturing the dynamics of memory. *IEEE Trans. on Knowledge and Data Engineering*, 35(10), 10085–10097.
- [3] Settles, B. & Meeder, B. (2016). A trainable spaced repetition model for language learning. *Proc. 54th Annual Meeting of the ACL*, 1848–1858.
- [4] Tabibian, B., Upadhyay, U., De, A., Zarezade, A., Schölkopf, B., & Gomez-Rodriguez, M. (2019). Enhancing human learning via spaced repetition optimization. *Proc. National Academy of Sciences*, 116(10), 3988–3993.
- [5] Puterman, M. L. (2014). *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons.